



REDBELLY

REDBRIDGE

Technical Guide · TASK-08: Existing Bridge Integration Guide

A working, testnet-deployed lock-and-mint bridge between Ethereum Sepolia and Redbelly Testnet, secured by a two-of-three multisig relayer network. Every transaction is independently verifiable on chain.

Live application	redbridge.test-hub.xyz
Bridge history API	api.redbridge.test-hub.xyz/api/bridge-history
Source chain	Ethereum Sepolia Chain ID 11155111
Destination chain	Redbelly Network Testnet Chain ID 153
Repository	github.com/0xDarkSeidBull/daotask8
Published article	dev.to/0xdarkseidbull

In this guide: what Redbridge is, the architecture, prerequisites, a step-by-step walkthrough, working code, gas costs, stuck-fund recovery, troubleshooting, verified transactions, and a look at existing bridge support for Redbelly Network.

What Redbridge Is

Redbridge is a lock-and-mint bridge. It uses the same fundamental pattern as most production bridges, including Wormhole and the LayerZero OFT bridges. This implementation is a single-asset-pair reference for the Redbelly ecosystem.

A user locks ETH on Sepolia. Three independent relayer processes watch for that lock, verify it, and once two of three agree, WETH.rb mints automatically on Redbelly Testnet at a one-to-one rate. There is no claim step. Nothing is required beyond the initial lock.

- Lock-and-mint architecture. The same base pattern as Wormhole and LayerZero, simplified to one asset pair.
- Two-of-three multisig relayer network. One compromised or offline signer cannot mint or halt the bridge alone.
- Fully automatic minting. No separate claim transaction. Mint fires the moment consensus is reached.
- Fully verifiable. Every lock and mint pair is checkable on Sepolia Etherscan and the Redbelly explorer.
- Live bridge history API. Every bridged transaction is queryable by anyone, not only the sender.
- Stuck-fund recovery built in. A self-serve support-ticket system with automatic already-bridged detection. See [Stuck-Fund Recovery](#).
- Testnet proven. Multiple real end-to-end transactions verified on chain, including a genuinely recovered stuck lock. See [Verified Testnet Transactions](#).

Roles

Role	Who	What they do
Bridge user	Anyone	Calls lock() on Sepolia. This is the only on-chain action required. There is no separate approve or claim step.
Relayer signers	Three pre-set wallets, configured at deploy via updateRelayerSigner()	Watch for Locked events independently, wait for confirmations, then call confirmMint(). The contract mints once two of three have called it.

Architecture

Why a relayer, and not a light client

A fully trustless bridge runs a light client of one chain inside the other. That is the approach LayerZero and Wormhole use at scale, backed by their own validator sets. For a testnet reference implementation, a permissioned relayer set is the standard simplified pattern. It demonstrates the full lock, verify, mint flow without a custom light-client implementation for Redbelly's consensus.

Why two-of-three, and not one-of-one

A single relayer is a single point of failure. If its key is compromised, arbitrary mints become possible. Requiring two of three independent signers to agree before a mint executes means one compromised or offline signer cannot mint or halt the bridge alone. This mirrors the multisig-relayer and guardian pattern used by several production bridges.

ETHEREUM SEPOLIA REDBELLY TESTNET

```
SepoliaLockVault.sol WETHBridged.sol (ERC-20)
  lock(recipient) confirmMint(...)
  payable, locks ETH 2-of-3 multisig gate
  | emits Locked event
  v
  watched by 3 independent relayer processes -----^
  each relayer verifies, then calls confirmMint()
  on the 2nd matching approval, WETHBridged mints
  WETH.rb 1:1 to the recipient, automatically
```

Repository Structure

```
daotask8/  
  contracts/  
    sepolia/  
      SepoliaLockVault.sol deployed on Sepolia, accepts and locks ETH  
    redbelly/  
      WETHBridged.sol deployed on Redbelly Testnet, 2-of-3 gated minter  
  scripts/  
    deploy-sepolia.js deploys SepoliaLockVault  
    deploy-redbelly.js deploys WETHBridged  
  test/  
    SepoliaLockVault.test.js  
    WETHBridged.test.js  
    EndToEndFlow.test.js full lock, approve, mint simulation  
  relayer/  
    relayer.js off-chain relayer process, run once per signer  
    db.js SQLite persistence layer for bridge history + support tickets  
    api.js read-only + support-ticket HTTP API  
    ecosystem.config.js pm2 process definitions, 2 relayer instances  
  hardhat.config.js  
  .env.example
```

Stack. Contracts in Solidity with Hardhat and ethers.js v6. The relayer runs on Node.js with SQLite, better-sqlite3, managed by PM2. The API is an HTTP service exposing live bridge history and the stuck-fund support-ticket system. Networks are Ethereum Sepolia as the source and Redbelly Testnet, chain ID 153, as the destination.

Prerequisites

Have these ready before starting.

- Node.js version 18 or later.
 - Sepolia ETH from a faucet, for example sepoliafaucet.com.
 - Redbelly Testnet RBNT to pay relayer gas.
 - An RPC provider whose free tier supports `eth_getLogs`. Check its block-range limit before committing to one. See Troubleshooting, item 1.
-

Step-by-Step Integration Walkthrough

1. Clone and install

```
git clone https://github.com/0xDarkSeidBull/daotask8.git
cd daotask8
npm install
```

2. Configure the environment

```
cp .env.example .env
```

Fill in `.env` with:

- `DEPLOYER_PRIVATE_KEY` deploys both contracts and becomes the initial contract owner.
- `SEPOLIA_RPC` see Troubleshooting. Do not use a plain public endpoint without checking its `eth_getLogs` range limit first.
- `REDBELLY_TESTNET_RPC` <https://governors.testnet.redbelly.network>
- `RELAYER_PRIVATE_KEY` and `RELAYER2_PRIVATE_KEY` two independent wallets, funded with Redbelly Testnet RBNT.

3. Deploy the contracts

```
npx hardhat run scripts/deploy-sepolia.js --network sepolia
npx hardhat run scripts/deploy-redbelly.js --network redbellyTestnet
```

Copy the deployed addresses into `LOCK_VAULT_ADDRESS` and `WETH_BRIDGED_ADDRESS` in `.env`.

4. Register the relayer signers

Run from the contract owner account.

```
npx hardhat console --network redbellyTestnet
const weth = await ethers.getContractAt("WETHBridged",
process.env.WETH_BRIDGED_ADDRESS);
await weth.updateRelayerSigner(0, "0xYourSigner1Address");
await weth.updateRelayerSigner(1, "0xYourSigner2Address");
await weth.updateRelayerSigner(2, "0xYourSigner3Address");
// signer 3 can stay unused if only running two signers
```

5. Run the relayers

Each signer gets its own env file and its own PM2 process.

```
cp .env .env.signer1 # set RELAYER_PRIVATE_KEY to signer 1, SIGNER_LABEL=signer1
cp .env .env.signer2 # set RELAYER_PRIVATE_KEY to signer 2, SIGNER_LABEL=signer2
pm2 start ecosystem.config.js
pm2 save
```

6. Verify and test

```
pm2 logs relayer-signer1 --lines 20 --nostream
```

Then call `lock()` on `SepoliaLockVault`, using the code in [Bridging: a Working Example](#) below. `WETH.rb` lands in the recipient's Redbelly wallet automatically, within a few minutes.

Bridging: a Working Example

There is no separate approve or claim step for the bridge user. Locking is the bridge transaction. Minting fires automatically once the relayer network reaches two-of-three consensus.

Setup

```
const { ethers } = require("ethers");
const sepoliaProvider = new ethers.JsonRpcProvider(process.env.SEPOLIA_RPC);
const wallet = new ethers.Wallet(process.env.USER_PRIVATE_KEY, sepoliaProvider);

const LOCK_VAULT_ABI = [
  "function lock(address redbellyRecipient) external payable",
  "event Locked(uint256 indexed nonce, address indexed sender, address indexed redbellyRecipient, uint256 amount, uint256 timestamp)",
];
const lockVault = new ethers.Contract(process.env.LOCK_VAULT_ADDRESS, LOCK_VAULT_ABI, wallet);
```

Lock ETH on Sepolia

```
async function bridgeETH(recipientOnRedbelly, amountEth) {
  const tx = await lockVault.lock(recipientOnRedbelly, {
    value: ethers.parseEther(amountEth),
  });
  console.log("Lock tx submitted:", tx.hash);
  const receipt = await tx.wait();
  const lockedEvent = receipt.logs
    .map((log) => { try { return lockVault.interface.parseLog(log); } catch { return null; } })
    .find((e) => e && e.name === "Locked");
  console.log("Locked with nonce:", lockedEvent?.args?.nonce?.toString());
  return lockedEvent?.args?.nonce;
}
bridgeETH("0xRecipientAddressOnRedbelly", "0.01");
```

Check mint status

Polling only. Minting itself is automatic.

```
const redbellyProvider = new ethers.JsonRpcProvider(process.env.REDBELLY_TESTNET_RPC);
const WETH_BRIDGED_ABI = [
  "function computeMintKey(uint256 sourceChainId, uint256 sourceNonce) public pure
  returns (bytes32)",
  "function mintRequests(bytes32) public view returns (address recipient, uint256
  amount, uint256 approvalCount, bool executed)",
];
const wethBridged = new ethers.Contract(process.env.WETH_BRIDGED_ADDRESS,
WETH_BRIDGED_ABI, redbellyProvider);
const SEPOLIA_CHAIN_ID = 11155111;

async function checkMintStatus(lockNonce) {
  const mintKey = await wethBridged.computeMintKey(SEPOLIA_CHAIN_ID, lockNonce);
  const state = await wethBridged.mintRequests(mintKey);
  return { approvals: Number(state.approvalCount), executed: state.executed };
}
// Poll every 15 seconds until executed is true.
```

Read `mintRequests()` directly instead of parsing it from event logs. Parsing `approvalCount` out of a decoded event argument proved unreliable in this implementation. See [Troubleshooting](#), item 4. Reading contract state directly is the authoritative source of truth.

Extending to ERC-20 tokens, if forking beyond native ETH

```
const TOKEN_ABI = ["function approve(address spender, uint256 amount) external returns
(bool)"];
const token = new ethers.Contract(TOKEN_ADDRESS, TOKEN_ABI, wallet);
const approveTx = await token.approve(LOCK_VAULT_ADDRESS, amountToBridge);
await approveTx.wait();
// Then call the vault's lock(token, amount, recipient) equivalent.
```


Gas Cost Analysis

Measured on Sepolia and Redbelly Testnet at the gas price in effect during testing, August 2026. Figures are gas units and native-currency cost only. USD cost varies with ETH and RBNT price and with network congestion.

Operation	Chain	Gas used	Notes
lock(), 0.001 ETH	Sepolia	~68,000	One SSTORE for the nonce plus event emission dominate the cost.
lock(), 0.012 ETH	Sepolia	~68,000	Gas cost is amount-independent. Same storage writes regardless of value locked.
confirmMint(), 1st approval	Redbelly Testnet	~85,000	Writes a new mintRequests entry, increments the approval count.
confirmMint(), 2nd approval, triggers mint	Redbelly Testnet	~140,000	Additional cost from the ERC-20 _mint() call and the MintExecuted event.

Transaction size does not affect gas cost. Only the number of distinct on-chain state changes does. This design has no batching discount. A production version handling high volume would benefit from a batched-mint pattern, where one confirmMint call approves many nonces at once.

Stuck-Fund Recovery: Support Tickets

A lock that never resolves, whether from a relayer-side bug or a user submitting a malformed recipient, previously had no visible recovery path beyond asking directly. Redbridge now exposes a self-serve support-ticket system, backed by the same bridge-history database, so a stuck or apparently-stuck transaction always has a next step.

- Open a ticket. A user submits their Sepolia lock tx hash and wallet address from the app's Contact Support / Reclaim flow. The ticket automatically attaches the lock's current on-chain status, so no manual lookup is required.
- Already-bridged detection is automatic. If the submitted transaction has, in fact, already minted successfully, the ticket is created and closed in the same request, tagged Already Bridged, with the destination Redbelly transaction attached as proof. Most "is my bridge stuck?" questions resolve themselves instantly, with no human in the loop.
- Genuinely stuck locks stay Active until manually resolved, with the on-chain lock status attached, so no context is lost between the user's report and whoever investigates it.
- Duplicate protection. Re-submitting the same transaction while a ticket is already open returns the existing ticket instead of creating a second one.
- My Tickets view. A wallet can list every ticket it has opened, split into Active and Solved, without needing to remember a ticket ID.

API

```
POST /api/support-ticket open a ticket (auto-resolves if already minted)
GET /api/support-ticket/:ticketId fetch a single ticket
GET /api/support-ticket/wallet/:address all tickets for a wallet, split Active / Solved
POST /api/support-ticket/:ticketId/resolve admin-only, marks a stuck lock as recovered
```

This closes the gap the original design left open: a stuck lock is no longer a silent failure. It's a tracked, resolvable ticket. The outcome, either already bridged or manually recovered, is documented and attached to a Redbelly transaction hash.

Troubleshooting

Real errors hit during development, with root cause and fix. Not hypothetical.

1. `eth_getLogs` fails with "archive request" or "requires a personal token"

Free public RPC endpoints, including publicnode and the Alchemy free tier, restrict block-range size on `eth_getLogs`. The Alchemy free tier caps it at 10 blocks per call.

Fix. Query in small chunks with a delay between calls. Keep `HISTORICAL_LOOKBACK_BLOCKS` small, between 300 and 500, unless on a paid tier.

```
async function getLogsChunked(contract, filter, fromBlock, toBlock, chunkSize = 10) {
  const allEvents = [];
  for (let start = fromBlock; start <= toBlock; start += chunkSize) {
    const end = Math.min(start + chunkSize - 1, toBlock);
    allEvents.push...(await contract.queryFilter(filter, start, end));
    await new Promise((r) => setTimeout(r, 400));
  }
  return allEvents;
}
```

2. Alchemy 429, "exceeded compute units per second capacity"

Multiple relayer signer processes hitting the same free-tier API key at once multiplies the request volume.

Fix. Add a delay between chunk requests, keep the lookback window small, or move to a paid RPC tier for production volume.

3. `SqliteError: database is locked (SQLITE_BUSY)` on relayer startup

Two relayer processes open the same SQLite file at close to the same moment.

Fix. Set an explicit busy timeout so SQLite retries instead of throwing.

```
const db = new Database(DB_PATH, { timeout: 10000 });
db.pragma("busy_timeout = 10000");
db.pragma("journal_mode = WAL");
```

4. `approvalCount` silently stored as 0 in the database

Parsing `approvalCount` from a decoded event argument occasionally returned undefined. `Number(undefined)` is NaN, and `better-sqlite3` silently coerces that to 0.

Fix. Never trust event-log parsing for state that is directly readable. Read `mintRequests(mintKey)` fresh from the contract instead.

```
const freshState = await wethBridged.mintRequests(mintKey);  
const approvalCount = Number(freshState.approvalCount); // authoritative
```

5. pm2 restart does not pick up .env changes

ecosystem.config.js reads process.env.* at file-load time, and Node caches that. pm2 restart does not re-execute the config file.

Fix. Use pm2 delete followed by a fresh pm2 start, or have each process load its own .env through a shell wrapper at startup.

```
#!/bin/bash  
set -a  
source /path/to/.env.signer1  
set +a  
exec node /path/to/relayer/relayer.js
```

6. Port already in use by an unrelated process

A new API server shows online in pm2 status, but every request returns 404. Another process was already bound to that port.

Fix. Run `lsof -i :<port>` to confirm, then pick a free port and update the reverse-proxy configuration.

7. Mint confirmation feels slow, about 60 to 90 seconds, on testnet

CONFIRMATION_BLOCKS is set to 5, waiting for 5 Sepolia block confirmations before relaying, plus a 15-second poll interval. On testnet, reorg risk is negligible compared to mainnet, so this margin is larger than it needs to be.

Fix. Drop CONFIRMATION_BLOCKS to 2 and the relayer's POLL_INTERVAL_MS to 5000 for a faster testnet experience. This cuts the wait to roughly 25 to 30 seconds, with still enough confirmation depth to stay safe against testnet reorgs. Going to 0 is not recommended, even on testnet. It removes the buffer entirely, so a reorged block could get relayed as final.

8. A stuck lock is permanently skipped as “possible reorg,” but it wasn’t a reorg

getLogsChunked() returned a partial, sometimes empty, event array on an RPC error without throwing. The retry loop logged the failure and silently returned whatever had been collected so far. verifyLockStillValid() then compared that against the expected lock and, on an empty result, assumed the lock had been reorged out. It had no way to tell an RPC hiccup apart from a chain that genuinely no longer contains the event, so it permanently skipped the lock, and the mint would never be attempted again without manual intervention.

Fix. Have getLogsChunked() throw once its retry budget is exhausted, instead of swallowing the error. Wrap the verifyLockStillValid() call in a try/catch: a thrown error means “ask again later” (re-queued with a 30-second backoff), and only a clean, successful query that

genuinely returns zero matching events counts as a reorg.

```
if (attempt >= 3) {
  console.error(`getLogs chunk [${start}-${end}] failed after 3 attempts`,
err.message);
  throw new Error(`getLogsChunked exhausted retries for [${start}-${end}]:
${err.message}`);
}
```

9. Verification window queries blocks that haven't been mined yet

verifyLockStillValid()'s query window was originalBlockNumber + CHUNK_SIZE (10 blocks ahead), but CONFIRMATION_BLOCKS was only 2. On a fresh lock, the relayer tried to verify against a block range that didn't exist on chain yet. This fired on every single lock, not just under RPC stress, and only resolved once the chain naturally caught up, roughly 90 to 100 seconds later, which the item 8 retry-queue patched over by design rather than fixing outright.

Fix. Cap the window's end at the chain's current block height, never past it.

```
const currentHead = await sepoliaProvider.getBlockNumber();
const windowEnd = Math.min(originalBlockNumber + CHUNK_SIZE, currentHead);
```

Verified Testnet Transactions

All transactions below were bridged in this testing cycle (19–20 August 2026) and are independently verifiable on the respective block explorers. Nonce #9 was genuinely stuck for roughly 22 hours due to the RPC-lag bug in Troubleshooting, item 8, and is included here specifically as proof that the fix, and the recovery path it enables, work end to end, not only in isolation.

Nonce	Sepolia lock tx	Redbelly mint tx	Amount	Status
#17	0xbef7522a...b91893a	0x815e7c58...f2345c	0.001 ETH	Minted
#16	0x84b13ca3...d05737	0x91f30864...d7774a	0.001 ETH	Minted
#15	0xc3d52399...19b657	0x1cca9aa1...b44a03	0.0011 ETH	Minted
#14	0x48a5267f...5381b7	0xebac3c36...082ffc	0.0012 ETH	Minted
#13	0x626c2af8...3ba878	0xe9a6e522...a48855	0.001 ETH	Minted
#9	0x2b370c4f...a7889e	0xe9e8f29f...4891ce	0.1 ETH	Minted, recovered from a stuck state (item 8)

Deployed contracts

Contract	Address
SepoliaLockVault	Sepolia 0x130d07624d00DF30A5C30C3D237fD5d99A3DdE11
WETHBridged	Redbelly Testnet 0x11Bef97d2d2063b41887A76403B852b52D151501

Live, queryable bridge history for all users, not only the addresses above: [GET api.redbridge.test-hub.xyz/api/bridge-history](https://api.redbridge.test-hub.xyz/api/bridge-history)

Existing Bridge Protocol Research

Whether major cross-chain protocols currently support Redbelly Network natively, checked against each protocol's own live chain-list documentation, not blog posts or older summaries.

Redbelly Mainnet is chain ID 151. Redbelly Testnet is chain ID 153. Confirm which one is the target before deploying. Mixing them up is a common mistake.

Protocol	Native Redbelly support	Source
LayerZero	Yes, on Mainnet. A dedicated Redbelly Mainnet deployment plus an OFT quickstart guide in the official docs.	docs.layerzero.network
Axelar	No confirmed support. Not listed in the chain-connectivity docs or the ecosystem page.	docs.axelar.dev
Wormhole	No. Absent from the official Supported Networks table as of July 2026, across all five products and roughly 40 chains listed.	wormhole.com/docs
Connext	Unconfirmed. New chains are added on request through Discord, not from a static list.	nxtp-docs.connext.network
Multichain	Not applicable. Discontinued in 2023.	

Four integrations beyond the original task list are directly relevant.

- Celer's cBridge added Redbelly Network support in August 2025. It bridges tokens between Ethereum, BNB Chain, and Redbelly.
- Redbelly's own official bridge, at reddex.io/bridge, is what Redbelly's own X account points users to for bridging USDC and USDT into the network, describing it as their official and exclusive DEX.
- Lucid Labs Bridge is what Redbelly's own developer docs recommend for the return path: to bring RBNT from a non-Redbelly chain to Redbelly, use Lucid Labs Bridge. It also has a Resend Transaction feature, for retrying a transfer that gets stuck mid-relay without contacting support first.
- Router Protocol's Nitro and Polymer are not bridges a user would pick directly, but they are the infrastructure behind the existing wrapped-RBNT deployments. Nitro powers the Solana wrapped-RBNT bridge. Polymer powers the Ethereum one.

This repository's bridge shows the minimal architecture a Redbelly integration needs when a protocol has no native support: a source-chain lock contract, a destination-chain mint

contract with multisig-gated minting, and an off-chain relayer network connecting the two. LayerZero is confirmed live on Redbelly Mainnet, so it could in principle replace this repository's custom relayer with its own DVN and Executor infrastructure, while keeping a similar lock-and-mint pattern. That is the natural next step for a mainnet deployment.

Roadmap

Complete

- ✓ Lock-and-mint core architecture (SepoliaLockVault and WETHBridged)
- ✓ Two-of-three multisig relayer network
- ✓ Automatic minting on consensus, no manual claim step
- ✓ Live bridge history API, queryable by any wallet
- ✓ End-to-end verification on Sepolia and Redbelly Testnet
- ✓ Chunked eth_getLogs scanning for free-tier RPC compatibility
- ✓ PM2-managed relayer processes with the env-reload fix applied
- ✓ Existing bridge protocol research (LayerZero, Wormhole, Axelar, Connex, cBridge)
- ✓ Stuck-fund recovery: support-ticket system with automatic already-bridged detection, duplicate protection, and admin resolution

Planned

- Batched-mint pattern for high-volume production use
- ERC-20 token support beyond native ETH
- Mainnet deployment, potentially on LayerZero DVN and Executor infrastructure

Built By

OxDarkSeidBull. Solo builder, Redbelly ecosystem contributor.

Also building: [LitDEX](#) · [BetsOnBlock](#)

License: MIT. See [LICENSE](#) in the repository.

Built for TASK-08 on the Redbelly DAO Task Board.